

Qivia EV — Documentation technique

Audience : CTO / équipe technique **Périmètre :** France **Date :** 2026-06-10 **Statut :** moteur de planification fonctionnel + proto « Copilote » (handoff navigation). App native en préparation.

Sommaire

1. Vue d'ensemble
 2. Fonctionnalités
 3. Modèle de données (champ par champ)
 4. Sources de données, fréquences de rafraîchissement, licences
 5. Abonnements, clés API et coûts réels
 6. Algorithmes
 7. Architecture logique & performance
 8. Conformité & juridique (RGPD/CNIL, radars)
 9. Sécurité
 10. Précision et limites du modèle
 11. Marche à suivre (roadmap technique)
 12. Annexes — constantes & formules
-

1. Vue d'ensemble

Le produit est un **moteur de planification d'itinéraire pour véhicules électriques** qui, à partir d'un point de départ, d'une destination et de l'état de charge réel d'un véhicule, calcule :

- l'itinéraire routier (trafic temps réel inclus) ;
- la **consommation énergétique** prédite, corrigée par la météo et le relief ;
- les **arrêts de recharge optimaux** (où, quand, combien de temps, à quel SoC) ;
- le **coût de recharge** et les **péages** ;
- le **SoC à l'arrivée** et la durée totale (conduite + charge) ;
- la **disponibilité** des bornes retenues (quasi temps réel) ;
- une **remise à la navigation** via deeplink Google Maps + QR (proto).

Le tout est conçu autour d'un **pipeline en 4 phases** : *routing* → *enrichissement & filtrage des bornes (en parallèle)* → *planification des arrêts* → *disponibilité & coût*.

2. Fonctionnalités

#	Fonctionnalité	État
F1	Itinéraire EV avec trafic temps réel	✓
F2	Modèle de consommation véhicule (courbe vitesse→conso, style de conduite)	✓
F3	Correction conso par météo (température + vent de face/dos)	✓
F4	Correction conso par relief (dénivelé + récupération en descente)	✓
F5	Sélection des bornes dans un corridor autour du trajet	✓
F6	Planner d'arrêts (modes Rapide / Économique)	✓
F7	Variantes avec / sans péage	✓
F8	Coût de recharge (tarification IRVE + référentiel opérateurs)	✓
F9	Disponibilité des bornes (quasi temps réel)	✓
F10	SoC arrivée, péages, durée totale	✓
F11	Handoff navigation (deeplink Google Maps + QR)	✓ (proto)
F12	Branchement télématique (SoC réel, 30 s)	→ SOON en cours
F13	Navigation embarquée temps réel + reroute auto	→ SOON roadmap
F14	Zones de danger (radars)	→ SOON roadmap

3. Modèle de données (champ par champ)

3.1 Profil véhicule

Champ	Type	Exemple (Tesla M3 LR)	Rôle
battery_kwh	float	75.0	Capacité batterie
aux_kw	float	1.0	Consommation auxiliaire (climatisation, etc.)
max_dc_kw	float	250.0	Puissance DC crête acceptée par le véhicule
consumption_curve	list[(km/h, kWh/km)]	(0,0.130)... (130,0.210)	Conso par palier de vitesse (route plate, climat doux)

Style de conduite appliqué en multiplicateur sur la courbe : **Souple 0.85 / Normal 1.0 / Dynamique 1.18**.

3.2 Point d'itinéraire (`RoutePoint`)

`lat`, `lng`, `km` (distance cumulée), `soc_pct` (SoC bridé ≥ 0 pour l'affichage), `speed_kmh` (vitesse moyenne de la section), `kwh_consumed_from_start` (**conso cumulée NON bridée** — sert au planner pour calculer les déficits « virtuels » au-delà de l'autonomie réelle).

3.3 Résultat d'itinéraire (`RouteResult`)

`provider`, `points[]`, `total_km`, `total_consumption_kwh`, `soc_at_arrival_pct`, `first_below_10pct`, `total_duration_s`, `total_toll_eur`, `avoids_tolls`.

3.4 Arrêt de recharge (`ChargingStop`)

`km`, `lat`, `lng`, `name`, `operator`, `city`, `power_kw`, `soc_arrival_pct`, `soc_leave_pct`, `kwh_added`, `charge_time_min`.

3.5 Plan de trajet (`TripPlan`)

`stops[]`, `feasible`, `reason`, `arrival_soc_pct`, `drive_time_s`, `charge_time_s`, `total_time_s`, `updated_points[]` (profil SoC après application des recharges), `mode`.

3.6 Bornes IRVE (champs exploités)

`lat / lng` (consolidated_latitude/longitude), `puissance_nominale`, `nom_operateur`, `nom_station`, `consolidated_commune`, `prise_type_*` (Type 2 / Combo CCS / CHAdeMO / E-F), `horaires`, `tarification` (texte libre).

3.7 Météo / Élévation / Disponibilité / Prix

- Météo (`WeatherSample`) : `temp_c`, `wind_speed_kmh`, `wind_dir_deg` (direction d'où vient le vent).
- Élévation : altitude (m) par coordonnée.
- Disponibilité : `status`, `n_total`, `n_available`, `n_occupied`.
- Prix : `price_per_kwh`, `total_eur`, `source` $\in$ {irve, operator, tier}.

3.8 Télémétrie (datalake client) — à brancher

Flux rafraîchi **30 s** : SoC (%), position GPS, vitesse/style, identifiant véhicule, identifiant conducteur. Remplace la saisie manuelle du SoC dans le proto.

4. Sources de données, fréquences, licences

Source	Donnée	Rafrâichissement	Clé	Licence / statut
HERE Routing API v8	itinéraire, polyline, conso, durée, péages, trafic	À la demande (trafic temps réel via <code>departureTime</code>)	Oui	Commerciale, freemium
TomTom Search/EV	disponibilité des bornes par connecteur	À la demande, cache 2 min	Oui	Commerciale, freemium
IRVE consolidé (data.gouv.fr)	~140 k bornes : position, puissance, opérateur, connecteurs, horaires, tarif	Jeu de données mis à jour quotidiennement ; CSV mis en cache ~24 h	Non	Licence Ouverte / Etalab (libre)
Open-Meteo	température, vent (vitesse + direction)	À la demande (conditions courantes)	Non	Gratuit non commercial ; commercial = payant
OpenTopoData (SRTM30m)	altitude	À la demande (statique)	Non	API publique rate-limitée (≈1 req/s, 1000/j) ; auto-hébergement pour le commercial
Photon (komoot)	autocomplétion d'adresses	À la demande	Non	Fair-use ; auto-héberger en commercial
Nominatim (OSM)	géocodage (fallback)	À la demande	Non	Politique stricte (1 req/s, UA obligatoire) ; auto-héberger/payant en prod
Google Maps (deeplink)	remise à la navigation	À la demande	Non	Deeplink gratuit (≠ embarquer Maps)
Télématique (datalake)	SoC, GPS, style, IDs	30 s	—	Infrastructure interne

5. Abonnements, clés API et coûts réels

À prévoir pour un produit commercial (≠ proto) :

- **HERE** — clé API. Free tier mensuel puis facturation à la requête au-delà. Vérifier les **droits de cache** des réponses et les quotas. *Bottleneck de latence (~5-8 s/requête).*

- **TomTom** — clé API. Free tier puis facturation. Utilisé pour la disponibilité (cache 2 min pour limiter le coût).
- **Open-Meteo / Nominatim / Photon / OpenTopoData** — ⚠️ **gratuits uniquement en usage non commercial / fair-use**. Pour un volume flotte : **plan payant ou auto-hébergement** (sinon rate-limit / blocage). À chiffrer comme un vrai poste.
- **Mapbox Navigation SDK** (phase native, cf. §11) — free tier **100 MAU + 1 000 trajets/mois** ; au-delà, modèle *metered* (MAU + trajet) ou *unlimited* (par MAU). Tarif entreprise au-delà de 5 000 MAU.
- **Télématique** — déjà disponible (datalake interne) ; pas d'abonnement tiers.

Recommandation CTO : isoler chaque dépendance derrière une interface (déjà le cas pour le routing HERE/TomTom) afin de pouvoir **basculer vers une version auto-hébergée** (Nominatim, OpenTopoData, Open-Meteo) le jour où le volume l'exige.

6. Algorithmes

6.1 Routing & consommation (HERE)

On envoie à HERE la **courbe de consommation** (`freeFlowSpeedTable` et `trafficSpeedTable` , paires vitesse→kWh/km), la charge initiale, la charge max et la conso auxiliaire. HERE renvoie des **sections** : polyline encodée (flexpolyline), longueur, **consommation kWh**, durée, et **péages** (EUR). Le trafic temps réel est activé via `departureTime = now` .

6.2 Distribution des points le long de la polyline

Pour chaque section, on décode la polyline puis on répartit km / kWh / SoC sur chaque sommet **proportionnellement à la distance haversine** entre points consécutifs :

```
frac_i      = (Σ segments jusqu'à i) / (Σ segments de la section)
km_i        = km_cumulé + section_km  · frac_i
kwh_i       = kwh_cumulé + section_kwh · frac_i
soc_i (%)   = max(0, (charge_initiale_kWh - kwh_i) / batterie · 100)
```

On stocke aussi `kwh_consumed_from_start` **non bridé** (peut dépasser l'autonomie réelle) pour le planner.

6.3 Style de conduite

`consumption_curve` multipliée par le facteur de style (0.85 / 1.0 / 1.18). Un style inconnu retombe sur 1.0.

6.4 Correction météo

Échantillonnage de **4 points** le long du trajet, requêtes Open-Meteo **en parallèle** (ThreadPool), puis interpolation linéaire (circulaire pour la direction du vent).

- **Facteur température** (réf. 20 °C) : $f_T = 1 + \max(0, 20 - T) \cdot 0.015 + \max(0, T - 25) \cdot 0.008$ (chauffage/batterie froide sous 20 °C ; clim au-dessus de 25 °C).
- **Facteur vent** (la part aéro croît en v^2) : $\text{part_aéro} = \min(0.85, 0.50 \cdot (v/100)^2)$; $v_{\text{eff}} = \max(1, v + \text{vent_de_face})$; $f_{\text{vent}} = 1 + \text{part_aéro} \cdot ((v_{\text{eff}}/v)^2 - 1)$ (neutre sous 30 km/h). Le **vent de face** est projeté via le cap (bearing) du segment : $\text{vent} \cdot \cos(\text{dir_vent} - \text{cap})$.

6.5 Correction relief (élévation)

Échantillonnage ≤ 60 points (limite OpenTopoData), interpolation linéaire de l'altitude sur tous les points. Énergie potentielle :

```
E_kWh = masse · g · Δh / 3.6e6    (masse = 1850 kg, g = 9.81)
montée : E / rendement_montée    (0.85)
descente: E · rendement_regen     (0.70, énergie récupérée)
```

Méta calculées : altitude min/max, dénivelé positif cumulé.

Recompose finale du SoC (par segment) : $\text{kwh_ajusté} = \text{kwh_base} \cdot f_T \cdot f_{\text{vent}} + E_{\text{relief}}$; SoC recalculé sur la conso cumulée.

6.6 Filtrage corridor (bornes proches du trajet)

1. **Pré-filtre bbox** : $\text{pad} = \text{corridor_km} / 111$ ($\approx 1^\circ \text{ lat} \approx 111 \text{ km}$).
2. **Sous-échantillonnage** de l'itinéraire (≤ 200 points).
3. **Haversine vectorisé numpy** sur la matrice ($n_{\text{bornes}} \times n_{\text{échantillons}}$) $\rightarrow$ distance min au trajet par borne ($\approx 50\times$ plus rapide qu'une boucle Python). On conserve `distance_to_route_km`, `km_along_route`, `soc_when_passing_pct`, et on garde les bornes à $\leq \text{corridor_km}$ (défaut 5 km).

6.7 Catégories de puissance (référentiel Avere-France)

Normale $< 7.4 \text{ kW}$ · Accélérée 7.4–22 · Rapide 22–50 · HPC 50–150 · Ultra-rapide ≥ 150 . Le planner ne retient par défaut que **Rapide / HPC / Ultra-rapide**.

6.8 Planner d'arrêts — *greedy lookahead*

On parcourt le profil SoC. Dès que le SoC passerait sous `min_soc_pct`, on cherche **en arrière** sur une fenêtre (`lookback_km`, défaut 200 km) la meilleure borne :

```
score = power_weight · min(puissance, max_dc)
      + position_weight · (km_borne / km_critique)
      - price_weight · prix_kWh          (mode éco uniquement)
```

On insère l'arrêt, on recharge jusqu'au **SoC cible de sortie**, et on **décale tout le profil restant** de l'énergie ajoutée. Répété jusqu'à arrivée avec marge, ou jusqu'à `max_stops` (12).

Puissance de charge effective (modèle de *taper*) — une borne rapide ne tient pas sa puissance crête sur toute la plage 10 → cible :

```
limité    = min(puissance_borne, max_dc_véhicule)
ratio     = limité / max_dc
facteur   = clamp( 0.95 - 0.55·ratio - max(0, (cible - 80)·0.01), ≥ 0.30 )
kW_effectif = limité · facteur
temps_charge_min = kWh_ajoutés / kW_effectif · 60
```

6.9 Modes Rapide / Économique

Param	fast	eco
SoC de sortie cible	70 %	85 %
SoC mini à l'arrivée	10 %	15 %
<code>power_weight</code>	0.6	0.3
<code>position_weight</code>	100	80
<code>price_weight</code>	0	200

Le mode **éco** fait une **recherche de budget** : il essaie des `price_weight` décroissants (400→40) et garde la 1^{re} solution faisable dont le temps total ≤ **1.2 × temps du mode rapide**.

6.10 Tarification / coût

1. Si le champ IRVE `tarification` contient un €/kWh explicite (regex défensive sur texte libre) → on l'utilise.
2. Sinon, **référentiel opérateurs** (`OPERATOR_PRICES`, ~30 opérateurs, prix ad-hoc carte bleue, réf. mi-2025).
3. Sinon, **défaut par palier de puissance** : 0.30 (≤22) / 0.45 (≤50) / 0.55 (≤150) / 0.65 (≤350) / 0.69 €/kWh. `coût_arrêt = prix_kWh · kWh_ajoutés`.

6.11 Disponibilité (TomTom)

Recherche de la borne par **proximité (≤ 300 m)** + indice de nom, puis appel `chargingAvailability` ; agrégation par connecteur (available / occupied / out_of_service / unknown). Statut global priorisé : disponible > occupée > HS > inconnue. **Cache 2 min**.

6.12 Remise à la navigation (proto)

Génération d'un **deeplink Google Maps Directions** (`api=1`, `dir_action=navigate`) avec la/les borne(s) en **waypoint**, plus un **QR** (API publique). Sur mobile, ouverture directe de Google Maps en navigation. ⚠ *Handoff unidirectionnel : aucune mise à jour possible après ouverture (cf. §11).*

7. Architecture logique & performance

Pipeline (par variante de trajet) :

```
Phase 1  fetch_route_here  ┌
Phase 1  (en parallèle)   ├── enrich_route (météo // + élévation)
                        ├── filter_corridor (bornes)
Phase 2  plan_trip (fast/eco)
Phase 3  disponibilité (// par arrêt) + coût
```

- Calculs **HERE x2 en parallèle** (avec / sans péage).
- Météo en **ThreadPool**, élévation en **batch**.
- Corridor IRVE **vectorisé numpy**.
- Caches : itinéraire/IRVE (24 h), disponibilité (2 min), géocodage (10 min).

Performance actuelle : ~13 s typiques, **dominés par la latence HERE (~5-8 s/requête)** — c'est le principal levier d'optimisation côté backend.

8. Conformité & juridique

⚠ Section critique pour un produit B2B flotte en France — à valider avec un conseil juridique.

8.1 RGPD / CNIL — géolocalisation des conducteurs

La télématique (position + style de conduite d'un **salarié**) est une **donnée personnelle**, encadrée par la CNIL : - **Base légale et information préalable** du conducteur. - **Consultation du CSE** (comité social et économique) pour le suivi des véhicules de salariés. - **Minimisation & durée de conservation** définies ; désactivation hors temps de travail. - **Droits** d'accès/rectification ; **finalité** strictement délimitée (pas de surveillance permanente). - Registre des traitements, analyse d'impact (AIPD) probable vu la géoloc.

8.2 Radars → zones de danger (obligation légale)

En France, **signaler la position exacte d'un radar fixe est interdit** (depuis 2012). Les apps doivent afficher des « **zones de danger** » **agrégées** (le compromis Coyote/Waze).

Conséquence produit : la donnée (open data Sécurité Routière) doit être **transformée en zones**, jamais affichée en points précis.

8.3 Licences des données

- IRVE : Licence Ouverte / Etalab (OK commercial).
 - OSM / Nominatim / Photon : attribution OSM + politique d'usage (auto-héberger en prod).
 - HERE / TomTom : licences commerciales, respecter les droits de cache/affichage.
-

9. Sécurité

- **Secrets** (clés HERE, TomTom, accès datalake) hors du code, en coffre / variables d'environnement chiffrées ; **rotation** régulière.
 - **Cloisonnement de la donnée conducteur** : la position d'un salarié ne doit être accessible qu'aux rôles légitimes (principe du moindre privilège).
 - **Transport chiffré** (TLS) de bout en bout, y compris flux télématique.
 - **Journalisation des accès** aux données de géolocalisation (traçabilité CNIL).
-

10. Précision et limites du modèle

- **Véhicule unique** aujourd'hui (Tesla M3 LR hardcodé). Une flotte réelle = N modèles ⇒ besoin d'une **bibliothèque de courbes de conso** (cf. roadmap).
 - Hypothèses physiques : masse 1850 kg, rendement montée 0.85, regen 0.70 — approximations.
 - Conso auxiliaire constante (`aux_kw`), même courbe réutilisée pour le trafic ralenti.
 - Tarifs de recharge = **estimations** (réf. mi-2025), pas un flux de prix temps réel.
 - Disponibilité = best-effort (matching par proximité), latence/qualité TomTom variables.
 - Météo échantillonnée sur 4 points, élévation sur ≤ 60 points (compromis latence/précision).
-

11. Marche à suivre (roadmap technique)

Jalon 0 — API du « cerveau » (*quelques jours*)

Exposer le pipeline existant (routing / enrichissement / corridor / planner / pricing) derrière une **API HTTP** (FastAPI). Aucune réécriture des algos : c'est un *wrapper*. Sert à la fois l'app native et tout autre client.

Jalon 1 — Télématic live (*1-2 jours, données déjà disponibles*)

Connecteur vers le **datalake (SoC, GPS, IDs ; 30 s)**. Le SoC réel remplace toute saisie manuelle. Comptes conducteur **déjà existants** ⇒ pas de chantier auth.

Jalon 2 — App native + navigation embarquée (*~2-3 semaines*)

App **mobile native** (ex. React Native) intégrant le **Mapbox Navigation SDK** (turn-by-turn embarqué, écran 100 % maîtrisé). La carte et l'itinéraire viennent du Jalon 0.

Pourquoi embarqué et pas deeplink : un deeplink Google Maps est unidirectionnel — une fois ouvert, impossible de mettre à jour la route. Le reroute automatique selon la conso réelle n'est possible que si la navigation tourne dans l'app.

Jalon 3 — Reroute anticipatif (*quelques jours une fois J1+J2 en place*)

Boucle : lecture conso réelle (30 s) → si dérive vs prévision, **recalcul de l'arrêt** et mise à jour **sans interruption** de la navigation embarquée. C'est le différenciateur cœur.

Jalon 4 — Zones de danger

Open data Sécurité Routière → **agrégation en zones** (conforme, cf. §8.2) → overlay sur la carte de navigation.

Jalon 5 — Cœur métier énergie (*à cadrer séparément*)

- **Réconciliation paiement** borne ↔ carte de recharge ; **€/km** en direct.
- **Dashboard flotte** : coût/km par véhicule, CO₂, reporting siège.
- **Bibliothèque multi-véhicules** (courbes de conso par modèle, ex. base type EVDB).
- **Robustesse navigation** : mode **hors-ligne** / perte réseau (tunnels, zones blanches).

Estimation consolidée : app native avec **nav embarquée + reroute télématic live** ≈ **3-4 semaines** (et non 2 mois), la télématic et les comptes conducteur étant **déjà disponibles**.

12. Annexes — constantes & formules

Véhicule de référence (Tesla M3 LR) : 75 kWh · aux 1.0 kW · DC max 250 kW. Courbe conso (km/h → kWh/km) : (0,0.130)(30,0.130)(50,0.140)(80,0.150)(100,0.165)(120,0.190)(130,0.210).

Constantes physiques : masse 1850 kg · g 9.81 · rendement montée 0.85 · regen 0.70.

Échantillonnage : météo 4 pts · élévation ≤ 60 pts · corridor ≤ 200 pts route, défaut 5 km.

Caches : IRVE/itinéraire 24 h · disponibilité 2 min · géocodage 10 min.

Paramètres planner : lookback 200 km · max 12 arrêts · catégories par défaut Rapide/HPC/Ultra-rapide.

Fin du document.